

Lab5

姓名：段光杰

学号：22307130301

日期：2025/5/14

实验目标：

MRET ECALL

MMU

实现过程

MRET和ECALL

根据lab6.pdf的标准来做，新增的关键点在于：首先，译码部分，在原有的csr指令译码的基础上添加：

```
if (is_csr) begin
    dataD.ctrl.csr = 1'b1;
    dataD.ctrl.csr_addr = csr_addr;
    unique case (funct3)
        F3_ECALL:
            unique case (dataF.raw_instr[31:20]) // funct12
                12'h000: begin dataD.ctrl.op = ECALL; dataD.ctrl.regwrite =
1'b0; end
                12'h001: begin dataD.ctrl.op = EBREAK; dataD.ctrl.regwrite =
1'b0; end
                12'h302: begin dataD.ctrl.op = MRET; dataD.ctrl.regwrite = 1'b0;
dataD.ctrl.csr = 1'b1; dataD.ctrl.csr_addr = CSR_MSTATUS; dataD.ctrl.csr_we = 1'b1;
end
            default: begin dataD.ctrl.op = UNKNOWN; end
    end
end
```

添加这部分就可以完成译码。csrfile的修改：

```
u1 ecalls;          // ECALL 异常到来
u1 mrets;           // MRET 返回
logic [62:0] ecode; // 异常号 (mcause[62:0])

assign ecalls = (op == ECALL) ? 1'b1 : 1'b0; // ECALL 异常
assign mrets = (op == MRET) ? 1'b1 : 1'b0; // MRET 返回
```

增加了两个信号，一个ecalls和mrets，这两个信号都是根据op来决定的，也就是指令，这里接收的信号是来自dataD的译码。这里的mode信号是新增的，用来表示三种模式，也就是用户模式、特权模式、机器模式。然后，ecode是异常发生的原因，这里的话稍微查表一下，写一个if赋值就可以完成。然后：

```

if (ecall) begin
    mepc          <= cur_pc; // 保存当前 PC
    nxt_pc        <= mtvec; // 设置异常返回地址
    mcause[63]    <= 1'b0; // 设置异常标志
    mcause[62:0]  <= ecode; // 设置异常号
    mstatus.mie   <= 1'b0; // 禁止中断
    mstatus.mpie  <= mstatus.mie;
    mstatus.mpp   <= mode; // 保存当前特权等级
    mode          <= 2'b11; // 设置特权等级
end
// **次优先：MRET 返回**
else if (mret) begin
    mstatus.mie   <= mstatus.mpie; // 恢复中断使能
    mstatus.mpie  <= 1;
    mstatus.mpp   <= 2'b00;
    mode          <= mstatus.mpp; // 恢复特权等级
    mstatus.xs    <= 0; // 恢复扩展状态
    nxt_pc        <= mepc; // 恢复返回地址
end

```

这里的话就是直接根据手册或者说参考的pdf来做的，这里有一个nxt_pc，是output信号，传回到core，同时，采用和jump信号同样的策略来完成跳转。此处不做更多赘述

关于这两个信号的stall和flush，实际上，在上次lab中已经实现了对于csr信号的stall和flush，对于这两个同样是csr指令的指令，无需修改。

mmu的实现：

关于虚拟内存和mmu这部分刚开始写实在一头雾水，后来才学习到，程序在刚开始执行的时候是使用机器模式，在完成必要的操作之后设置satp寄存器，切换为用户模式，才开始使用mmu，也就是说，在机器模式下，mmu是完全空转无用的。

mmu的输入输出

以下是mmu的实现代码，通过直接接受i_addr和i_valid，也就是指令地址和指令有效，之前的做法是在core中发出ireq请求，现在是通过mmu向总线发送ireq请求，再获得iresp回到mmu模块，对于dreq，mmu模块直接截断memory模块发出的tmp_dreq请求，然后将dreq发送给总线，最后接收到有效的（已通过三级页表索引）dresp，在将tmp_dresp发送给memory模块。简而言之，对于memory模块来说，它并不知道自己的数据被截断了，所以，memory模块不需要任何的修改。

mmu的状态机

根据实验材料的提示，来构建索引三级页表的一个状态机。

首先，对于机器态（privilegeMode）为3的情况，不需要通过页表，直接访问物理地址，那么，需要两个状态，第一个，是MMU_NOP，

```

MMU_NOP: begin
    if (i_valid | tmp_dreq.valid) begin
        if ((satp.mode == 8) & (privilegeMode != 3)) begin
            state <= PAGE_1;
            dreq.valid <= 1;
            dreq.addr <= {8'b0, satp.ppn, addr[38:30], 3'b0};
            dreq.size <= MSIZE8;
            dreq.strobe <= 'b0;
            dreq.data <= 'b0;
            stall_mmu <= 1'b1;
        end
        else begin
            state <= MMU_DIRECT;
            if (tmp_dreq.valid) begin
                dreq <= tmp_dreq;
            end
            if (i_valid) begin
                ireq.valid <= 1;
                ireq.addr <= i_addr;
            end
        end
    end
end
end

```

如果发现是机器模式，那么直接跳转到向真实物理地址访问状态MMU_DIRECT，然后完成取数据。如果不是，则进入PAGE_1，接下来，通过dreq完成三级页表索引真实物理地址。

```

MMU_DIRECT: begin
    if (dreq.valid & dresp.data_ok) begin
        state <= MMU_GAP;
        tmp_dresp <= dresp;
        tmp_iresp <= 'b0;
        dreq.valid <= 0;
        stall_mmu <= 1'b0;
    end
end

```

接下来，需要冲刷一下tmp_dresp和tmp_iresp，那么又需要一个新状态MMU_GAP，这个状态用于冲刷上一条指令的数据

```

MMU_GAP: begin
    tmp_dresp <= 'b0;
    tmp_iresp <= 'b0;
    if (ireq.valid) state <= MMU_DIRECT;
    else state <= MMU_NOP;
end

```

然后是三级页表索引，一级接一级，最后到达MMU_DIRECT，完成取数据

```

PAGE_1: begin
    if (dresp.data_ok) begin
        state      <= PAGE_2;
        dreq.valid  <= 1;
        dreq.addr   <= {8'b0, dresp.data[53:10], addr[29:21],
3'b0}}; dreq.size  <= MSIZE8;
        dreq.strobe <= 'b0;
        dreq.data   <= 'b0;
    end
end

PAGE_2: begin
    if (dresp.data_ok) begin
        state      <= PAGE_3;
        dreq.valid  <= 1;
        dreq.addr   <= {8'b0, dresp.data[53:10], addr[20:12],
3'b0}};

        dreq.size  <= MSIZE8;
        dreq.strobe <= 'b0;
        dreq.data   <= 'b0;
    end
end

PAGE_3: begin
    if (dresp.data_ok) begin
        state <= MMU_DIRECT;
        if (tmp_dreq.valid) begin
            dreq.valid <= tmp_dreq.valid;
            dreq.addr  <= {8'b0, dresp.data[53:10],
tmp_dreq.addr[11:0]}};

            dreq.size  <= tmp_dreq.size;
            dreq.strobe <= tmp_dreq.strobe;
            dreq.data   <= tmp_dreq.data;
            ireq.valid  <= 0;
        end
        else if (i_valid) begin
            dreq.valid <= 0;
            ireq.valid <= 1;
            ireq.addr  <= {8'b0, dresp.data[53:10],
i_addr[11:0]}};
        end
    end
end
end

```

到这里为止，本次实验就完成了

问题 输出 调试控制台 终端 端口 1

```
Using /home/gyro/arch/arch-2025/ready-to-run/riscv64-nemu-interpreter-so for difftest
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'clint' at [0xa2000000, 0xa200ffff]
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
dump wave 0-999999 to /home/gyro/arch/arch-2025/build/1747197847.fst...
The first instruction of core 0 has committed. Difftest enabled.
xv6 kernel is booting
[WARNING] difftest store queue overflow
kinit ok
procinit ok
trapinit ok
plicinit ok
userinit ok
Return from init! Test passed
^Z
[1]+  Stopped                  make test-lab5 VOPT="--dump-wave"
gyro@Gilo:~/arch/arch-2025$
```

上板：

串口调试助手 + TCP/UDP

66 execute_data_t dataE, dataE_next;
67 memory_data_t dataM, dataM_next;
68 writeback_data_t dataW;
69
70
71
72
73
74 端口名: COM6
75 波特率: 9600
76
77 数据位: 8
78 校验位: None
79 停止位: 1
80
81 < RI DSR CTS DTR RTS
82
83 关闭
84
85
86

Button press detected!

xv6 kernel is booting
kinit ok
procinit ok
trapinit ok
plicinit ok
userinit ok
Return from init! Test passed

接收设置:
☐ 将接收保存到文件
☐ 十六进制显示
☐ 自动断帧 20 ms
☐ 脚本 Add Timestamp
保存数据 清空数据

发送: 0 接收: 391 - 42 复位计数

问题 输出

```
Using /home/gyro/arch/arch-2025/ready-to-run/riscv64-nemu-interpreter-so for difftest
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'clint' at [0xa2000000, 0xa200ffff]
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
dump wave 0-999999 to /home/gyro/arch/arch-2025/build/1747197847.fst...
The first instruction of core 0 has committed. Difftest enabled.
xv6 kernel is booting
[WARNING] difftest store queue overflow
kinit ok
procinit ok
trapinit ok
plicinit ok
userinit ok
Return from init! Test passed
^Z
[1]+  Stopped                  make test-lab5 VOPT="--dump-wave"
gyro@Gilo:~/arch/arch-2025$
```

实验难点

由于代码的规范，以及状态机的规范，上板十分顺利完成了。本次实验遇到的麻烦在于：难以调试，波形图太过于冗长，有一个很长的循环，刚开始我甚至看不见波形图，使用display来调试。

第二个问题，找不到指令地址，由于使用虚拟地址，导致无法找到真正出错的指令是那一行，只能通过波形图来调试，导致了很多无用功（比如ecall指令错了但是一直不知道错的地方是一条ecall指令）